F&IP Case study: F&IP, an F&I data platform

Michael Hanson · demonstration project · all data synthetic · numbers from pipeline run `20261026T183720Z`

THE PROBLEM

A company that administers the protection products dealers sell in the finance office (vehicle service contracts, GAP, maintenance plans) lives on its reports. Dealers, finance, compliance and executives each need numbers they can trust, every morning. The usual warehouse grew one urgent request at a time: reports read the admin system directly, the same dealer exists under three ids, one layer stores claims negative and another positive, and nobody can say whether a report still matches the source until someone complains.

WHAT I BUILT

- **The whole chain, end to end.** A synthetic admin system, a daily dealer feed, CRM and admin-API intake with data contracts and quarantine, a layered dbt warehouse (raw, staging, intermediate, marts; 175 models built in the measured run), accounting-grade facts, state reserve filings, dealer statements and per-dealer packets.
- **Gates that block publishing.** Sixteen diagnostics, a source-to-mart reconciliation and a publish gate. If one breaks, yesterday's reports stay up and nothing new goes out.
- **Reports people can read.** A static report site with Excel downloads, nine department hubs, and a three-line plain-English explanation under every tile and chart.
- **A Power BI path.** A semantic model generated from one metrics file, a Power BI project over a reconciled PostgreSQL copy of the marts, and dealer row-level security from one access map.
- **The old way, rebuilt on purpose.** A legacy-style warehouse built from the same synthetic rows, so the same ten business questions can be measured both ways.

MEASURED RESULTS

Result	Value	What is behind it
Rows stored in the warehouse	`14.9M`	14,954,790 rows in 166 DuckDB tables and snapshots, all layers (11,320,528 in the marts reports read); 59 views not counted
dbt build nodes passed	`928 of 929`	pipeline build: 175 models, 4 snapshots, 26 seeds, 722 data tests, 2 unit tests; 0 failed; 1 known data-quality warning (warn_rv_missing_chassis_vin, 30 rows); 11 of 11 gate tests after
Python tests passed	`779`	0 failed, 2 skipped of 781 run; 2 slow end-to-end tests not in this record; recorded 2026-10-26
Full rebuild time	`9 min 54 s`	10 of 10 steps ok, source data to published site, one command (594.67 s)
Gate breaks	`0`	16 of 16 diagnostics passed; 36 of 37 reconciliation checks tied out, 1 documented waiver; 3 of 3 publish gates ok
Legacy answers wrong	`10/10 → 0`	old warehouse wrong or failed on 10 of 10 business questions; new platform: 0 errors or timeouts
Slow legacy query	`961.9 ms → 44.6 ms`	Q07 (Attributes for a provider-scoped population), the benchmark's slow query, legacy vs new, same server and rows; legacy Q10 timed out at 60 s (new: 32.2 ms)
Dealer packets	`150`	one packet per reportable dealer, filtered to that dealer's rows only

Five of the ten legacy-vs-modern questions, same synthetic rows, same server (benchmark of 2026-10-26):

- One dealer's in-force book: legacy 230 contracts, modern 473 (the dealer exists under several ids).
- Contracts by product, latest plan year: legacy counted 50,371, modern 59,749 (a stale product table, inner-joined).
- Paid claims: legacy -$18,883.27, modern $6,187,027.54 (claim signs differ between layers).
- Provider-scoped lookup (the slow-query case): legacy 961.9 ms, modern 44.6 ms (a union of every layer is rebuilt for each lookup).
- Vehicle match by VIN: the legacy lookup timed out at 60 s; its fallback join matched 78.58%. Modern: 100.00% in 32.2 ms on an enforced key.

The modern side is slower on 2 of 10 questions (Q05 and Q09); there the fix is the right answer, not speed.

HOW I WOULD RUN IT AT A REAL COMPANY

1. **Weeks 1-2: listen and map.** Read-only access to the admin system, an inventory of every report and who uses it, and one owner per metric definition. No dashboards yet.
2. **Land and reconcile first.** Copy sources untouched into raw, then tie row counts and dollars from source to raw to marts on every run. A report is only trusted once its numbers tie out.
3. **Gates before go-live.** The checks that matter most (match rates, fan-out, claim sign, demo rows, freshness) block publishing; every run is logged against a 7 AM target.
4. **Dev, UAT, prod.** Every change builds in UAT, is diffed against prod, and ships only when the change ticket explains every difference.
5. **One definition, many audiences.** Metrics live in one file that generates the Power BI models; each department gets a hub with its questions, its pages and who to ask when a number looks wrong.
6. **Then scale the platform.** The same dbt project already has Snowflake and Postgres targets, an Airflow DAG and row-level security scripts, written and ready for the company's accounts.

HONEST NOTES

- **Built with AI coding agents.** I directed them: I wrote the design contract, gave each agent one role on a data team's org chart, and checked every report it returned against the tests and run artifacts before accepting it. The agents wrote most of the code; I decided what shipped.
- The pytest record behind the table had 0 failing of 781 collected tests, shown as is.
- The legacy in-force book at the 2023 cutover is not migrated; reconciliation reports it as a documented exclusion, not a pass. The Snowflake and Airflow pieces are written but not executed here.
- All data is synthetic. No real company's data, names or layouts are used.

Code and README: https://github.com/MichaelH117/fi-data-platform